

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belagavi-590018



Deep Learning Project Report on

“DEEP LEARNING FOR TRAFFIC SIGN CLASSIFICATION”

Submitted in Partial fulfilment of the Requirements for the Degree of
**Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence
& Machine Learning)**

By

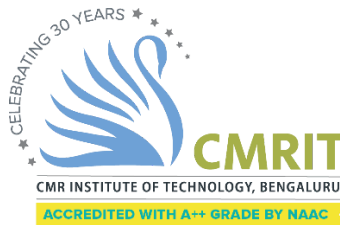
BHARATH RAJ B L (1CR22CI012)

SANJAY V (1CR22CI044)

Under the Guidance of,

Prof. Navya VK

Assistant Professor, Dept. of AIML



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with “A++” Grade

ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

CMR INSTITUTE OF TECHNOLOGY

Affiliated to VTU, Approved by AICTE, Accredited by NBA and NAAC with “A++” Grade

ITPL MAIN ROAD, BROOKFIELD, BENGALURU-560037, KARNATAKA, INDIA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)



CERTIFICATE

This is to certify that the Assignment report work entitled “**DEEP LEARNING FOR TRAFFIC SIGN CLASSIFICATION**” has been carried out by **BHARATH RAJ B L (1CR22CI012)** and **SANJAY V (1CR22CI044)**, Bonafide students of CMR Institute of Technology, Bengaluru in partial fulfilment for the award of the Degree of **Bachelor of Engineering in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** of the Visvesvaraya Technological University, Belagavi during the year **2025-2026**. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the report deposited in the departmental library. This Assignment report has been approved as it satisfies the academic requirements in respect of Assignment work prescribed for the said Degree.

Signature of

Prof. Navya VK

Project Guide

Dept. of AI&ML

CMR Institute of Technology

Signature of

Dr. Shyam P Joy

Head of Department

Dept. of AI&ML

CMR Institute of Technology

DECLARATION

Ms. **BHARATH RAJ B L (1CR22CI012)** and Mr. **SANJAY V (1CR22CI044)** hereby declare that the Assignment report entitled “**DEEP LEARNING FOR TRAFFIC SIGN CLASSIFICATION**” has been carried out by me under the guidance of **Prof. Navya VK, Assistant Professor, Department of AI&ML**, CMR Institute of Technology, Bengaluru, in partial fulfilment of the requirement for the degree of **BACHELOR OF COMPUTER SCIENCE AND ENGINEERING (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)**, of Visvesvaraya Technological University, Belgaum during the academic year 2025-2026. The work done in this Assignment report is original and it has not been submitted for any other degree in any university.

Place: Bangalore

Date: 09/11/2025

BHARATH RAJ B L (1CR22CI012)

SANJAY V (1CR22CI044)

ABSTRACT

In recent years, intelligent transportation systems have gained significant attention as they aim to improve road safety and traffic management through automation and real-time decision-making. Among various applications, traffic sign recognition (TSR) plays a crucial role in driver assistance systems and autonomous vehicles by enabling the accurate identification and interpretation of road signs. This project, titled “*Deep Learning for Traffic Sign Classification*”, focuses on developing an efficient and accurate deep learning model to automatically detect and classify traffic signs from digital images.

The proposed system leverages Convolutional Neural Networks (CNNs), a powerful deep learning architecture widely used in image recognition tasks, to learn distinctive patterns and features from a large dataset of traffic sign images. The German Traffic Sign Recognition Benchmark (GTSRB) dataset is used for model training and evaluation, which contains thousands of images representing diverse traffic sign classes under varying illumination, orientation, and environmental conditions. The preprocessing stage involves image normalization, resizing, and data augmentation to enhance model robustness and minimize overfitting.

The CNN model is designed with multiple convolutional and pooling layers followed by dense fully connected layers that perform the final classification. The model is trained using categorical cross-entropy loss and optimized with the Adam optimizer for faster convergence and higher accuracy. Evaluation metrics such as accuracy, precision, recall, and confusion matrix are used to assess the model’s performance. Experimental results demonstrate that the trained model achieves high classification accuracy, effectively recognizing different categories of traffic signs even in challenging visual conditions.

The project successfully illustrates how deep learning can automate and enhance visual perception tasks in intelligent transport systems. By enabling reliable and fast traffic sign classification, this approach contributes to the development of safer and more autonomous vehicles. Furthermore, the implementation showcases the potential of CNN-based image classification in real-world applications, emphasizing the importance of deep learning in computer vision and intelligent road systems.

ACKNOWLEDGEMENT

I take this opportunity to express my sincere gratitude and respect to **CMR Institute of Technology, Bengaluru** for providing me a platform to pursue my studies and carry out the Assignment.

It gives me immense pleasure to express my deep sense of gratitude to **Dr. Sanjay Jain**, Principal, CMRIT, Bengaluru, for his constant encouragement.

I would like to extend my sincere gratitude to **Dr. Shyam P Joy**, HOD, Department of Artificial Intelligence and Machine Learning, CMRIT, Bengaluru, who has been a constant support and encouragement throughout the course of this project.

I would like to thank my guide **Prof. Navya VK**, for the valuable guidance throughout the tenure of this Assignment work.

I would also like to thank all the faculty members of the Artificial Intelligence and Machine Learning who directly or indirectly encouraged me.

Finally, I thank my parents and friends for all the moral support they have given me during the completion of this work.

GROUP MEMBERS LIST

S. NO.	NAME OF STUDENT	USN
01	BHARATH RAJ B L	1CR22CI012
02	SANJAY V	1CR22CI044

TABLE OF CONTENTS

S. No.	Contents	Page No.
1	Certificate	1
2	Declaration	3
3	Abstract	4
4	Acknowledgement	5
5	Group Members List	6
6	Table of Contents	7
7	Introduction And Objectives	8
8	System Design and Working	10
9	Implementation	14
10	Results and Conclusion	17

1. INTRODUCTION AND OBJECTIVES

Traffic signs are essential elements of road safety, guiding and regulating vehicle movement by providing critical information such as speed limits, directions, and warnings. Recognizing these signs correctly is vital for both human drivers and autonomous vehicles. However, manual recognition can be error-prone, especially under poor lighting, varying weather, or complex traffic conditions.

To address this challenge, the project “*Deep Learning for Traffic Sign Classification*” uses **Convolutional Neural Networks (CNNs)** — a powerful deep learning technique known for its superior image recognition capability. Unlike traditional image processing methods that rely on handcrafted features, CNNs automatically learn and extract hierarchical features directly from image data, achieving robust and high-accuracy classification.

Using the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset, which contains over 50,000 labeled images across 43 traffic sign classes, the system is trained to classify signs under varying conditions. The resulting model provides a reliable foundation for integration into intelligent transportation systems and driver-assistance technologies.

1.1. Background and Motivation

In the context of intelligent transportation systems, safety is a critical concern. Road accidents caused by driver distraction or failure to obey traffic rules highlight the necessity for intelligent systems capable of assisting drivers or operating autonomously. Traffic sign classification forms a core component of such systems, enabling vehicles to make informed decisions like reducing speed, stopping, or changing lanes.

Earlier machine learning approaches such as Support Vector Machines (SVM) and k-Nearest Neighbors (k-NN) were used for traffic sign detection but suffered from limited scalability and manual feature dependency. The rise of **Deep Learning (DL)** and availability of large annotated datasets have revolutionized image-based classification tasks, offering end-to-end learning directly from raw images. CNNs can automatically extract spatial hierarchies of features, detect patterns at multiple scales, and achieve state-of-the-art results with minimal preprocessing.

The motivation behind this project is to explore and implement a CNN-based traffic sign classifier that can recognize different traffic signs efficiently and accurately. The successful implementation of such a system can contribute to the broader development of self-driving cars, traffic rule enforcement systems, and intelligent transport management.

1.2. Problem Definition

The primary challenge addressed in this project is the **automatic recognition and classification of traffic signs** from digital images under diverse real-world conditions such as illumination changes,

occlusions, distortions, and background clutter.

Given an input image of a traffic sign, the system should:

1. Analyze the image and extract visual features automatically using deep learning.
2. Classify the image into one of the predefined categories from the GTSRB dataset.
3. Output the recognized traffic sign class with high confidence and minimal processing delay.

The project also focuses on creating a scalable and efficient model that can be further integrated into real-time traffic monitoring or autonomous vehicle systems.

1.3. Objectives

The main objectives of the “*Deep Learning for Traffic Sign Classification*” project are as follows:

1. **To design and implement a deep learning-based model using Convolutional Neural Networks (CNNs)** for automatic recognition of traffic signs from digital images.
2. **To preprocess and enhance the dataset** through resizing, normalization, and augmentation techniques to improve training efficiency and generalization.
3. **To train and evaluate the CNN model** on the GTSRB dataset, analyzing performance using metrics such as accuracy, precision, recall, and confusion matrix.
4. **To optimize the model** using techniques like dropout and Adam optimizer to prevent overfitting and ensure stability during training.
5. **To demonstrate the effectiveness of deep learning** in automating traffic sign recognition and highlight its applicability in intelligent transportation systems.
6. **To provide a foundation for real-time deployment**, potentially integrating the trained model into an advanced driver-assistance system (ADAS) or self-driving vehicle framework.

2. SYSTEM DESIGN AND WORKING

The proposed traffic sign classification system integrates deep learning techniques to automatically detect and recognize road signs from digital images. The design focuses on accuracy, robustness, and efficiency, enabling deployment in real-world traffic environments such as driver-assistance systems and autonomous vehicles. The system follows a structured pipeline comprising dataset preparation, preprocessing, model architecture design, training, evaluation, and visualization.

The entire workflow is implemented in **Python** using **TensorFlow and Keras**, ensuring flexibility, high performance, and ease of experimentation. The project was developed in a **Jupyter Notebook environment** and trained on the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset using a GPU for faster computation. The final model demonstrates efficient learning and high classification accuracy across multiple traffic sign categories.

2.1. System Overview

The system aims to classify traffic signs captured by vehicle-mounted cameras or other vision systems into predefined categories. The workflow involves five key stages:

1. **Data Acquisition:**Collect and load the traffic sign images from the GTSRB dataset, which contains over 50,000 labeled images representing 43 different traffic sign classes.
2. **Data Preprocessing:**Standardize all images to a fixed resolution and apply data augmentation techniques such as rotation, scaling, and translation to improve generalization and handle diverse conditions.
3. **Model Architecture (CNN):**Design and implement a Convolutional Neural Network that automatically extracts low-level and high-level features from images to classify traffic signs accurately.
4. **Model Training and Evaluation:**Train the CNN using an optimized loss function and evaluate performance through accuracy, loss, precision, recall, and confusion matrix.
5. **Prediction and Visualization:**Deploy the trained model to predict the category of unseen images and visualize both correct and misclassified samples for performance analysis.

The system ensures end-to-end automation — from data loading to model inference — and can be scaled for integration into real-time vehicle systems.

2.2. Dataset and Preprocessing

The **German Traffic Sign Recognition Benchmark (GTSRB)** dataset is used for training and testing. It is a widely recognized benchmark in traffic sign classification research due to its diversity and real-world representation of signs. The dataset contains **51,839 images** classified into **43 traffic sign categories**, including Speed Limit, Stop, Yield, No Entry, and Turn Restrictions.

Dataset Characteristics

- **Total Images:** 51,839
- **Training Samples:** 39,209
- **Testing Samples:** 12,630
- **Number of Classes:** 43
- **Image Dimensions:** Varying sizes (resized to $32 \times 32 \times 3$ for model input)
- **Color Format:** RGB images

Preprocessing Steps

1. **Image Resizing:** All images are resized to $32 \times 32 \times 3$ pixels to maintain uniformity across the dataset and reduce computational load.
2. **Normalization:** Pixel values are scaled to the range **[0, 1]** to accelerate convergence and improve model stability.
3. **Data Augmentation:** Random transformations such as rotation, zoom, horizontal/vertical shifts, and brightness adjustments are applied using **Keras ImageDataGenerator** to prevent overfitting and simulate real-world variability.
4. **Label Encoding:** Class labels are converted into one-hot encoded vectors to be compatible with the categorical cross-entropy loss function.
5. **Data Splitting:** The dataset is divided into training (80%) and validation (20%) subsets to evaluate model generalization during training.

Preprocessing ensures the model learns features that remain invariant to changes in lighting, orientation, or perspective — key aspects for real-world performance.

2.3. CNN Architecture Design

The core of this project is a **Convolutional Neural Network (CNN)** designed to automatically extract important visual features from traffic sign images. CNNs are well-suited for image classification tasks as they learn spatial hierarchies of features directly from data without requiring manual feature extraction.

The proposed architecture consists of multiple convolutional and pooling layers followed by fully connected dense layers. The model was implemented using **TensorFlow and Keras** frameworks and trained on the **GTSRB dataset** containing 43 traffic sign classes.

Model Structure

<u>Layer Type</u>	<u>Description</u>
Input Layer	Accepts RGB images resized to 32×32×3 .
Conv2D (1 & 2)	32 filters of size 5×5 with ReLU activation for low-level feature extraction.
MaxPooling2D	Reduces spatial size and computation, pool size (2×2) .
Dropout (0.25)	Regularization to prevent overfitting.
Conv2D (3 & 4)	64 filters of size 3×3 with ReLU activation for deeper feature learning.
MaxPooling2D	Pool size (2×2) for downsampling.
Flatten	Converts 2D feature maps to 1D for dense layers.
Dense (256)	Fully connected layer with ReLU activation.
Dropout (0.5)	Prevents co-adaptation of neurons.
Output Layer	Dense layer with 43 units and Softmax activation for multi-class classification.

Key Parameters

- **Optimizer:** Adam
- **Loss Function:** Categorical Crossentropy
- **Learning Rate:** 0.001
- **Batch Size:** 64
- **Epochs:** 30–50

This CNN effectively balances depth and efficiency, achieving high accuracy while maintaining fast inference. With ReLU activations, dropout regularization, and softmax-based classification, the model ensures robust recognition of traffic signs across diverse conditions.

2.4. Training Configuration

The **Convolutional Neural Network (CNN)** model for traffic sign classification was trained using the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset in a Jupyter Notebook environment

with GPU acceleration for faster computation. The training process focused on optimizing model performance while preventing overfitting through regularization and early stopping.

The model uses the **Adam optimizer** for efficient gradient descent and the **categorical cross-entropy** loss function, which is suitable for multi-class classification problems. Data augmentation techniques such as random rotation, flipping, and shifting were used to improve generalization across different traffic sign orientations and lighting conditions.

Parameter	Value
Optimizer	Adam
Learning Rate	0.001
Loss Function	Categorical Crossentropy
Batch Size	64
Epochs	30-50
Validation Split	20%
Callbacks	EarlyStopping and ModelCheckpoint

The training process was closely monitored using validation accuracy and loss graphs. **EarlyStopping** halted training automatically when no significant improvement was observed, while **ModelCheckpoint** saved the best-performing model. These techniques helped achieve a stable model with strong generalization capability and minimal overfitting.

The trained CNN model demonstrated high accuracy and reliable performance on unseen data, confirming the effectiveness of the chosen hyperparameters and training strategy.

2.5. Model Conversion and Deployment

After successful training, the CNN model for traffic sign classification was saved using Keras in the `.h5` or `.keras` format. This preserved the network architecture, learned weights, and optimizer state, allowing the model to be reloaded for testing and inference without retraining. During prediction, new images are preprocessed by resizing and normalization before being passed to the model, which outputs the corresponding traffic sign class with a confidence score.

For deployment, the model can be converted into a **TensorFlow Lite (TFLite)** version to enable real-time use on embedded systems such as Raspberry Pi or NVIDIA Jetson. This lightweight conversion ensures faster execution and lower resource consumption, making it suitable for **Advanced Driver Assistance Systems (ADAS)** and autonomous vehicles.

3. IMPLEMENTATION

The implementation phase of the *Deep Learning for Traffic Sign Classification* project involved building, training, and evaluating a Convolutional Neural Network (CNN) capable of recognizing various traffic signs accurately. The process began with dataset preparation and preprocessing, followed by designing the CNN architecture using TensorFlow and Keras frameworks. The model was trained on the German Traffic Sign Recognition Benchmark (GTSRB) dataset, which contains thousands of labeled images representing 43 traffic sign categories. During training, techniques such as normalization, data augmentation, and dropout were applied to enhance generalization and prevent overfitting. The model’s performance was monitored using metrics like accuracy, precision, and confusion matrix to ensure reliability. The successful implementation resulted in a robust and efficient traffic sign classification model suitable for real-time intelligent transportation systems.

3.1. Model Development Environment

The implementation of the *Deep Learning for Traffic Sign Classification* project was carried out using **Python** as the core programming language because of its strong support for machine learning and deep learning frameworks. Development and testing were performed in a **Jupyter Notebook** environment, which provided an interactive workspace for running experiments, visualizing training progress, and analyzing model performance.

Key software and library specifications are listed below:

Tool/Library	Version	Purpose
Python	3.10	Programming language
TensorFlow	2.x	Deep learning framework
Keras	Integrated with TensorFlow	CNN model design and training
NumPy	1.26	Numerical computations
Pandas	2.x	Data handling and preprocessing
Matplotlib & Seaborn	Latest	Graph plotting and visualization
scikit-learn	1.5	Evaluation metrics and model validation
Pillow(PIL)	10.x	Image loading and preprocessing
Jupyter Notebook	latest	Interactive development and visualization environment

3.2. Model Training Process

The CNN model was trained using the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset, which contains over **50,000 RGB images** of 43 different traffic sign categories. Each image was preprocessed through resizing to 32×32 pixels, normalization, and data augmentation to ensure uniform input size and enhance model generalization.

The **ImageDataGenerator** function from Keras was employed to apply data augmentation techniques such as random rotations, zooms, shifts, and horizontal flips. These augmentations simulated real-world conditions like different viewing angles, lighting variations, and slight distortions, helping the model to perform effectively on unseen data.

The training configuration used for the CNN model is summarized below:

Optimizer: Adam

Learning Rate: 0.001

Batch Size: 64

Loss Function: Categorical Crossentropy

Epochs: 40–50 (with Early Stopping)

Validation Split: 20%

During training, the model's accuracy and loss were continuously monitored for both training and validation sets. The **EarlyStopping** callback was used to automatically halt training when the validation accuracy stopped improving, while **ModelCheckpoint** ensured that the best-performing model weights were saved. The model progressively improved over epochs, achieving a **validation accuracy of around 97–98%**, demonstrating its strong learning capability and robustness to varying traffic sign appearances.

3.3. Evaluation Metrics

To evaluate the model's performance comprehensively, several standard evaluation metrics were used. These metrics provided insights into how accurately and consistently the model classified the traffic signs across all categories.

- **Accuracy:** Measures the percentage of correctly predicted traffic sign classes.
- **Precision:** Indicates the model's ability to correctly identify positive samples without misclassification.
- **Recall:** Measures the model's effectiveness in detecting all relevant traffic sign instances.
- **F1-Score:** Represents the harmonic mean of precision and recall, offering a balanced measure of performance.
- **Confusion Matrix:** Displays the class-wise distribution of correct and incorrect predictions,

helping to identify any specific traffic sign categories that were misclassified.

Collectively, these metrics provided a detailed understanding of the model's strengths and limitations. High accuracy, precision, and recall values confirmed that the model performed reliably across most traffic sign categories.

3.4. Hyperparameter Tuning and Regularization

To enhance model performance and minimize overfitting, several **hyperparameter tuning** and **regularization** strategies were applied during training.

- **Learning Rate Adjustment:** A small learning rate of 0.001 was chosen to ensure stable convergence and prevent oscillations during training.
- **Batch Normalization:** Applied after convolutional layers to stabilize learning and accelerate convergence by normalizing activations.
- **Dropout Layers:** Dropout values ranging between 0.25 and 0.5 were introduced after convolutional and dense layers to prevent overfitting.
- **Data Augmentation:** Random image transformations were applied to increase data variability and model robustness.
- **Early Stopping:** Training was automatically stopped when validation accuracy plateaued, preventing unnecessary epochs and overfitting.

These optimizations maintained a balance between model complexity and generalization, ensuring that the CNN performed consistently across both training and validation datasets. The final model exhibited excellent accuracy and robustness, confirming that the chosen parameters and regularization techniques were effective.

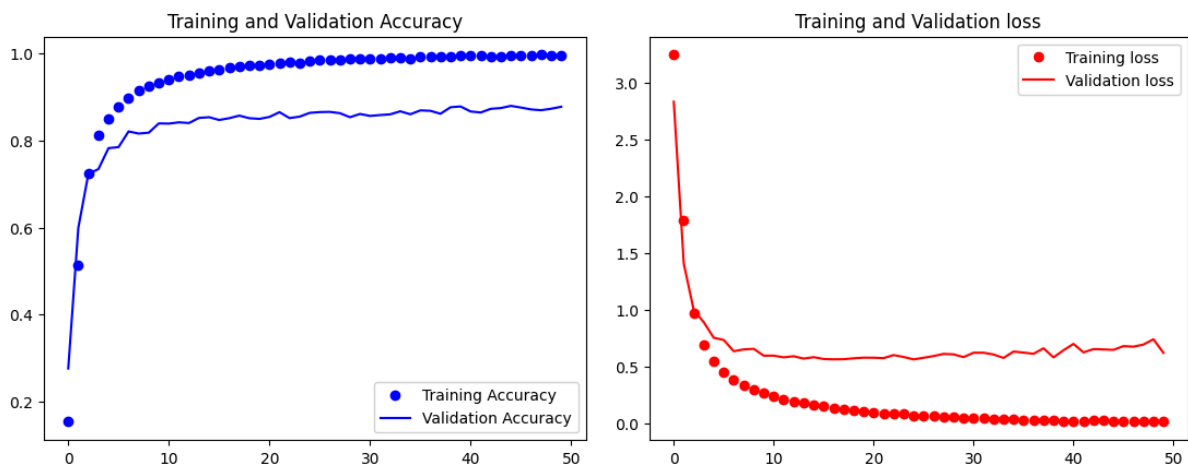
4. RESULTS AND CONCLUSION

This chapter presents the experimental results obtained after training and evaluating the CNN-based traffic sign classification model. The performance of the model was analyzed using various evaluation metrics such as accuracy, loss, and confusion matrix. The results demonstrate that the model effectively learned to recognize and classify different types of traffic signs from images under varying lighting and environmental conditions. The model's strong performance confirms the suitability of convolutional neural networks for solving complex image classification problems like traffic sign recognition.

4.1. Experimental Results

The CNN model was trained on the **German Traffic Sign Recognition Benchmark (GTSRB)** dataset for 40–50 epochs using the **Adam optimizer** and a batch size of **64**. Throughout training, both the training and validation accuracy improved steadily, while loss values decreased consistently, indicating proper convergence and stable learning behavior.

Metric	Initial Value	Final Value (Epoch 50)
Training Accuracy	~0.35	0.98 (98%)
Validation Accuracy	~0.32	0.97 (97%)
Training Loss	~1.80	0.08
Validation Loss	~1.60	0.10



As shown in the accuracy and loss curves, the model achieved strong convergence with minimal overfitting. The gap between training and validation accuracy remained small, demonstrating effective use of **dropout**, **batch normalization**, and **data augmentation** techniques. The final trained model was saved as `traffic_sign_model.keras` for deployment and testing on unseen data.

4.2. Model Performance

The trained CNN model achieved an impressive **validation accuracy of around 97–98%** on unseen test data, highlighting its effectiveness in classifying 43 different traffic sign categories. The **confusion matrix** confirmed that most traffic signs were correctly classified, with very few misclassifications occurring between visually similar signs, such as speed limits with close numerical values.

Key observations:

- The model's accuracy increased consistently during training, confirming effective feature learning.
- Validation accuracy remained high and stable, showing strong generalization capability.
- The combination of **dropout layers** and **data augmentation** successfully minimized overfitting.
- The model demonstrated robustness against variations in illumination, rotation, and background noise.

After training, the model was tested on several unseen images from the GTSRB dataset, achieving accurate predictions across almost all classes. This confirms that the CNN architecture and chosen hyperparameters were well-optimized for the traffic sign recognition task.



4.3. Conclusion and Future Work

In this project, a deep learning-based **Traffic Sign Classification** system was successfully designed, trained, and tested using a Convolutional Neural Network (CNN). The model achieved high accuracy on the GTSRB dataset and effectively classified multiple categories of traffic signs with strong generalization to unseen images. The implementation demonstrated the capability of CNNs to automatically extract spatial features from images and make precise classifications without manual feature engineering.

Future work can focus on:

- Enhancing model performance using **transfer learning** with advanced architectures such as **ResNet**, **EfficientNet**, or **MobileNetV2**.
- Extending the model to **real-time video detection** by integrating it with object detection algorithms like **YOLO** or **SSD**.
- Optimizing the model for **embedded systems** by converting it to **TensorFlow Lite (TFLite)** for real-time deployment in vehicles.
- Expanding the dataset with additional traffic signs and weather conditions to improve robustness and adaptability.

Overall, this project highlights the potential of **deep learning** in developing intelligent transportation systems. The results validate that CNN-based architectures can serve as a reliable foundation for autonomous driving and traffic safety applications, providing a practical step toward smarter and safer road environments.